

Spring 2025

Bot Detection in Social Media using GraphSage and BERT

Abhishek Deshmukh

Follow this and additional works at: https://scholarworks.sjsu.edu/etd_projects



Part of the [Other Computer Engineering Commons](#)

Recommended Citation

Deshmukh, Abhishek, "Bot Detection in Social Media using GraphSage and BERT" (2025). *Master's Projects*. 1465.

DOI: <https://doi.org/10.31979/etd.wb6h-3yd6>

https://scholarworks.sjsu.edu/etd_projects/1465

This Master's Project is brought to you for free and open access by the Theses and Graduate Research at SJSU ScholarWorks. It has been accepted for inclusion in Master's Projects by an authorized administrator of SJSU ScholarWorks. For more information, please contact scholarworks@sjsu.edu.

Bot Detection in Social Media using GraphSage and BERT

A Project

Presented to

The Faculty of the Department of Computer Science

San José State University

In Partial Fulfillment

of the Requirements for the Degree

Master of Science

by

Abhishek Deshmukh

December 2024

© 2024

Abhishek Deshmukh

ALL RIGHTS RESERVED

The Designated Project Committee Approves the Project Titled

Bot Detection in Social Media using GraphSage and BERT

by

Abhishek Deshmukh

APPROVED FOR THE DEPARTMENT OF COMPUTER SCIENCE

SAN JOSÉ STATE UNIVERSITY

December 2024

Dr. Teng Moh	Department of Computer Science
--------------	--------------------------------

Dr. Melody Moh	Department of Computer Science
----------------	--------------------------------

Dr. Navrati Saxena	Department of Computer Science
--------------------	--------------------------------

ABSTRACT

Bot Detection in Social Media using GraphSage and BERT

by Abhishek Deshmukh

This project details a novel bot detection system developed to battle the ever-changing challenge of disinformation, misinformation, and other bot-generated content. The methodology employed in this project combines the text-based analytical strength of BERT (Bidirectional Encoder Representations from Transformers) with the strength of GraphSage (Graph Sample and Aggregation) for analyzing network structures. The project concatenates BERT and GraphSage vectors to create an 896-size feature embedding with a rich blend of network and text features. This project employs a Support Vector Machine to process the concatenated embeddings, as SVM works well with high-dimensional data. This project was evaluated on two datasets, namely Cresci-15 and Twibot-22. This model outperformed all other models on the Cresci-15 dataset with an accuracy of 98.68%. Despite the challenges, the model had an accuracy score of 74.62% on the Twibot-22 dataset which still outperformed some state-of-the-art models. The model results highlight an efficient and scalable system that can handle large-scale datasets with intricate network structures.

ACKNOWLEDGMENTS

I am deeply grateful to Dr. Teng Moh for his mentorship and guidance throughout this project. The work done would not have been achievable without his guidance. Additionally, I appreciate the commitment and invaluable insights of the committee members Dr. Melody Moh and Dr. Navrati Saxena, who have generously contributed their time and expertise to this endeavor.

TABLE OF CONTENTS

CHAPTER

1	Introduction	1
1.1	Related Work	2
1.2	Report Overview	3
2	Background	4
2.1	GraphSage	4
2.2	BERT	5
2.2.1	DistilBERT	5
2.3	Datasets	6
2.3.1	Twibot dataset	6
2.3.2	Cresci Dataset	6
2.4	Experiments	7
2.4.1	Considerations	7
2.4.2	Experiments on Cresci-15	8
2.4.3	Experiments on Twibot-22	9
3	Implementation	10
3.1	Data Preprocessing	11
3.1.1	User Data Preprocessing	11
3.1.2	Tweet Data Preprocessing	12
3.2	DistilBERT Embeddings from tweet data	13
3.3	Feature Extraction from user data	13

3.4	Graph Construction	14
3.5	GraphSage Embeddings	15
3.6	Concatenation of Embeddings	17
3.7	SVM Classifier	17
3.8	Evaluation	18
3.8.1	Execution Details	18
4	Results	19
4.1	Results	19
4.1.1	Cresci-15 Results	19
4.1.2	Twibot-22 Results	20
5	Conclusion and Future Work	22
5.1	Future Work	23
	LIST OF REFERENCES	24
	APPENDIX	
	Generating Tweet embeddings using DistilBERT	27

CHAPTER 1

Introduction

As the digital landscape grows evermore, the spread of hate speech, misinformation, and disinformation has the ability to reach almost everybody on these platforms. The easiest way to disseminate such information is through the use of automated programs, i.e, “bots”. Bots on social media are developed in a way to disguise themselves as real users and are programmed to share information that is intended to manipulate social discourse.

Social media platforms characterized by their vast reach and immediacy make them very susceptible to bot-driven manipulation. Misinformation propagated by bots is usually carefully designed in a way to blur the lines between fact and fiction, which makes discerning truthful information quite difficult. Due to the ever-changing nature of social media, there arises a more demanding need to have defenses against bot-driven content. As the landscape changes, the methodologies employed to detect and deter such agents should too keep up with the complexities.

To deter bot-driven content generation, traditional ML models have been applied to varying success. As bots continue to evolve in complexity, classical models which infer using statistical data or content analysis have not been able to reliably keep up. To address these issues, various Graph Machine Learning techniques and Natural Language Processing techniques have been employed that have enabled a deeper understanding of bot dynamics.

This project introduces a novel technology to detect bots implemented using Bidirectional Encoder Representations from Transformers (BERT) and GraghSage in conjunction with a Support Vector Machine (SVM). BERT is employed to understand textual based features and GraphSage to understand the network characteristics. These text and network features are then combined to form a single composite feature

set. This feature set is then processed by the SVM in order to classify each account as a real user or a “bot”.

1.1 Related Work

In the field of classification, bot classification poses a significant and nuanced challenge. Advancements in this field have developed various ways to counteract bots using combinations of machine learning techniques along with NLP techniques to result in robust systems. This section highlights some of the state-of-the-art technologies that use Graph Machine Learning to detect bot-driven activities on complex social network structures.

BGSRD [1] stands for Social Bots Detection via Fusing BERT and Graph Convolutional Networks, introduced a novel technique to detect bots on social networks. Their methodology operates by constructing a text document graph using BERT embeddings generated from account posts. These are then converted to a dense joint representation using specific words in special categories. Finally, a GCN (Graph Convolutional Network) is employed to process the text document graph. After which they are classified using a Graph Attention Network [2]. BGSRD highlights the importance of combinatorial techniques which employ pulling features from network characteristics as well as text based features resulting in robust bot detection strategies.

BotRGCN introduced in [3], is a premier example for showcasing the efficacy of Graph Machine Learning models for identifying bots on social networks. The BotRGCN model constructs a heterogeneous graph representing the Twitter social network. It works by mapping the users to individual nodes and then drawing edges which signify different relationships such as follower, mentions or retweets. BotRGCN incorporates node features from user data such as tweet history, verification status, user description and follower/following count. On the heterogeneous graph described

above, R-GCNs are employed to get detailed user representations, and then they are classified using a prediction head. The results in BotRGCN highlight the robustness of using Graph Based models to understand user representation in a social network.

BGSRD and BotRGCN are two of the premier methodologies when compared to various baseline models on the Twibot and Cresci datasets.

1.2 Report Overview

The structure of the report is as follows:

- Section 2 highlights the key technologies used in the project, describes the datasets employed, and explains the challenges involved.
- Section 3 describes the implementation of the methodology.
- Section 4 presents the results of the methodology along with a comparison with current baselines and state-of-art models.
- Section 5 summarizes the project while also providing insights into improvements and possibility of further research.

CHAPTER 2

Background

This section describes the technologies employed in the methodology while also providing key insight behind the motivation. Challenges encountered along with their considerations are also described in this section.

2.1 GraphSage

GraphSage [4] is a key technology employed in this paper. GraphSage provides an extension to traditional GCNs [5] by introducing an inductive framework that allows the model to generate representations on unseen data. GraphSage has the ability to learn representations from by aggregating and sampling each individual node’s neighbors. GraphSage can then be used to provide a vector representations for each node. This is the key idea utilized in this project. GraphSage is used to generate vector embeddings for each user which is represented as node. This is further detailed in Section 3.

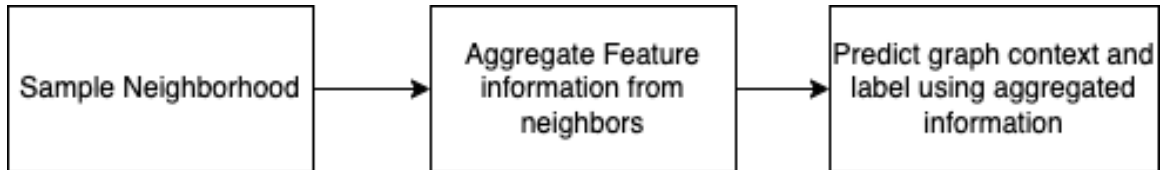


Figure 1: GraphSage Aggregation Approach

The above figure describes the traditional use case of GraphSage. It begins by sampling the neighboring nodes followed by aggregating the information from the neighbors. Which is then followed by predicting graph context to label each node. In this project, we do not use the prediction head but rather use it to generate embeddings which provide insights about the node features and the graph structure.

2.2 BERT

Bidirectional encoder representations from transformers [6], better known as BERT is another key technology employed in this project. BERT is a seminal paper in the field of Natural Language Processing, its key innovation is learning contextual information of a token in a sentence from both sides i.e left to right, and right to left. BERT employs a novel training approach which uses a masked language model. BERT’s effectiveness is highlighted across the domain of Natural Language Processing.

Similar to how GraphSage is employed, this project employs BERT to generate text representations of each user’s tweets in the form of 768-size vector embeddings. As highlighted by the findings in BGSRD [1], BERT adds a great deal of contextual information extracted from user text data which leads to a richer feature set for model training. Using BERT in conjunction with Graph based models enhances the classification capabilities of the models even further.

2.2.1 DistilBERT

DistilBERT [7] is a lightweight version of BERT which reportedly is rated to be 40% smaller and 60% faster than the standard BERT model but maintains 97% of the accuracy. This is another key technology employed in this project due to challenges highlighted in the considerations section.

DistilBERT’s achieves the above characteristics by employing knowledge distillation. This process enables DistilBERT to achieve comparable efficacy to BERT despite the reduced size. Due to the size and efficiency advantages, DistilBERT was chosen over BERT for the Twibot dataset due to enormous size. DistilBERT enabled the implementation of the methodology on the much larger set. This consideration is explained further in the experiments section.

2.3 Datasets

This section describes the datasets employed and their significance in the field of bot detection and misinformation classification on social media.

2.3.1 TwiBot dataset

The Twibot-20 dataset [8] and its successor the Twibot-22 dataset [9] are considered to be the premier benchmarking resources for research in the field of bot detection and misinformation classification.

The Twibot-20 dataset was created by applying a breadth-first strategy from selected seed users on Twitter. This dataset sets itself apart by its sheer size and depth. Twibot-20 dataset contains about 200 recent user tweets, each user’s metadata, and user followers and following accounts. The authors annotated the data by crowdsourcing reviews from Twitter users and by examining tweet patterns and originality. This enabled the creation of a well labelled dataset which contains labels for not just “bots” but on the types of bots.

The Twibot-22 dataset is an improved version of the Twibot-20 dataset where limitations for scale and annotations were addressed. The Twibot-20 dataset included about 23,000 annotated accounts while the Twibot-22 dataset includes more than 1 million accounts and more than 80 million tweets in total. The Twibot-22 dataset also includes more temporal features such as account creation data which adds to a more richer feature set. Twibot-22 has become a standardized benchmark for testing the effectiveness of Graph Based and Transformer based approaches for bot detection.

2.3.2 Cresci Dataset

The Cresci-15 dataset [10] has been a crucial resource to benchmark models on a smaller scale. The dataset was created by Stephano Cresci et al, with the intention of providing a holistic view of the social network. The dataset contains only about 5000

accounts with almost a 50/50 split between real users and bot accounts, this resulted in a well balanced dataset. The accounts were also manually annotated to result in an accurately labelled dataset.

The smaller size and scale of the dataset enables testing various methodologies, it includes various user metadata such as user follower/following counts, retweeted user lists, and temporal data regarding the account. The Cresci dataset is employed in this project to test and configure baselines for the model before applying and comparing them on the Twibot dataset. The types of baselines employed are highlighted in more detail in the Experiments section.

2.4 Experiments

This section details the experiments conducted, the challenges encountered, and highlights key considerations while undertaking the methodology.

2.4.1 Considerations

BERT as mentioned previously is a large scale pre trained model, this project employs it to generate embeddings for each tweet of each user. When considering the Twibot-22 dataset, it includes more than 1 million users and over 80 million tweets. It is very computationally expensive to generate these embeddings, not quite possible with local machine limitations. Due to this challenge, this project employs DistilBERT which is much lighter and much less computationally intensive with very minor drawbacks. The dual implementation also enables to verify the methodology even further which is detailed in the Results section.

Method	Processing Time (hours:mins)
BERT	$\approx 320 : 23$
DistilBERT	$\approx 100 : 55$

Table 1: Time duration for Preprocessing All user tweets in Twibot 22

The above table shows the processing duration required to generate embeddings

for each users tweets with BERT and DistilBERT. BERT requires over 300 hours on the local machine while DistilBERT offering a huge improvement still requires over 100 hours. These timelines are unfeasible for the project timelines as well as for any real world applications.

The massive size of the Twibot dataset and the sheer computing power required for BERT models, it was unfeasible to generate embeddings for all tweets per user. To overcome this issue, only the 15 most recent tweets were processed. This was decided by the following factors:

- Recent tweets show the current behaviour, as bots can have changing behaviours.
- Only allowing the most recent tweets, this makes the system quite efficient for processing.
- It enables the system to be more scalable as we can increase the number of tweets processed after increasing compute resources, this could be done horizontally or vertically.

Method	Processing Time (hours:mins)
BERT	$\approx 75 : 12$
DistilBERT	$\approx 13 : 40$

Table 2: Time duration for Preprocessing 15 tweets per user in Twibot 22

From the above table, we can identify that the process is much more feasible and falls within the project timeline. BERT still remains unfeasible due to the lack of compute resources on the local machine as it requires about 100 hours to process but DistilBERT only requires about 13 hours. This enables the testing and analysis of the methodology even with the lack of significant computational power.

2.4.2 Experiments on Cresci-15

Cresci-15 dataset being much smaller in size than the Twibot dataset as showcased in the Background section, it only includes 5301 user accounts and about 2 million

tweets. This enables the methodology to process the dataset using both the full BERT model, and the DistilBERT model.

The size also enabled the use of all of tweets per user for both BERT models. This enabled the analysis of the efficacy of the proposed methodology and provided a few baselines. It also showcased if there was any significant reduction in accuracy or F1 scores by using DistilBERT.

There were 2 main experiments conducted for Cresci-15:

- BERT + GraphSage + SVM
- DistilBERT + GraphSage + SVM

In both cases, the GraphSage and SVM models were identical to ensure consistent analysis.

2.4.3 Experiments on Twibot-22

As highlighted in the Considerations section, the sheer size and compute power required to process the Twibot dataset resulted in modified processes for the methodology.

Two experiments were conducted on the Twibot dataset:

- GraphSage + SVM (without text embeddings)
- DistilBERT + GraphSage + SVM (limited to 15 tweets per user)

The methodology was applied once without using any text embeddings but rather just the GraphSage embeddings and an SVM. This established a baseline methodology for the experiments and allowed the project to verify if the BERT embeddings were providing any value.

The methodology was applied a second time with DistilBERT embeddings which were limited to 15 tweets per user as described in the considerations section previously.

For the experiments, GraphSage and SVM models remain consistent throughout.

CHAPTER 3

Implementation

This section describes the implementation details of the methodology. The diagram below showcases the project workflow. The diagram highlights each step beginning with dataset preprocessing and ending with evaluation.

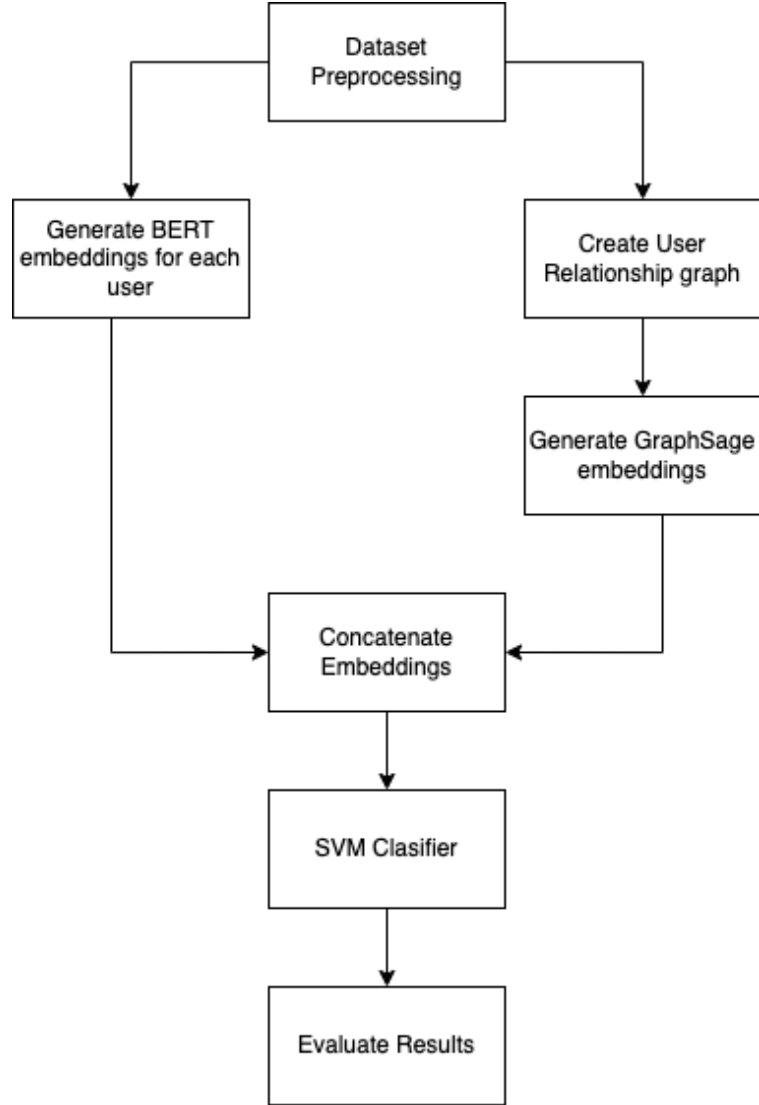


Figure 2: Methodology workflow

The methodology begins with dataset preprocessing, this involves preparing the dataset to be eligible for inputs to BERT models and GraphSage models. After

preprocessing the data, the textual data which are the tweets of each user are then processed using BERT, meanwhile an edge index matrix is created from the dataset. This is then used to construct the graph which as an input to the GraphSage model to generate graph embeddings. Next step is where both embeddings are concatenated to form a single vector which contains both textual features and network based features. Finally these embeddings are processed using an SVM classifier. The SVM results are then evaluated for each experiment, the accuracies, and F1 scores are compared to other state-of-art models on both datasets.

3.1 Data Preprocessing

This methodology provides consistent data preprocessing across both datasets, Cresci-15, and Twibot-22. Both datasets offer most of the same information for each user, which enables the data processing pipeline to be streamlined. The data preprocessing step is divided into two different parts namely User data processing, and Tweet data processing. The steps are isolated to streamline the process as the inputs for BERT and GraphSage require entirely different parts of the dataset.

3.1.1 User Data Preprocessing

The user data processing begins with first cleaning the data. This involves:

- Removing repeated rows to prevent skewed results
- Handling missing values by removing rows where values in critical fields such as user ID, tweet count, followers count, following count, favorites count and creation date are missing.

The next step is the creation of the edge indices required for graph construction. This data essentially contains the adjacency matrix in a coordinate list format, where each line represents an edge linking two nodes (users). The steps include:

- Extracting relationships: Identify and extract user relationships from the user

data. This includes followers, friends and interactions such as retweets and mentions. The goal is to construct a homogeneous graph, so the type of relation does not matter in this case. Every relation is treated the same. If a relation exists, it is considered to be an edge between the nodes.

- Mapping to indices: Users are mapped to unique indices that serve as nodes in the graph. The user ID is replaced with a sequential index to facilitate easier processing.
- Building edge list: An edge list is created where each row represents a connection between two users. This list includes source and destination indices derived from user relationships.
- Conversion to tensor format: Finally, the edge list is converted into a tensor format suitable for use with GraphSage.

3.1.2 Tweet Data Preprocessing

Since the quality of text data directly impacts the performance of BERT and DistilBERT, cleaning tweet data is a fundamental step. This involves:

- Text normalization: Lowercasing the text and removing numbers to reduce the variability in the text.
- Removing noise: Stripping out URLs, hashtags, mentions, and special characters that do not contribute to the understanding of the tweet's context.
- Handling emojis and symbols: Removing emojis and symbols that are not standard ASCII characters.
- Stop words removal: Eliminating common words (e.g., "and", "the") that are not useful for analysis to focus on meaningful words.

Specifically for the Twibot-22 dataset, only the most recent 15 tweets are considered for each user as highlighted in the Background section. For Cresci, all tweets of

each user are considered.

The preprocessing steps detailed above are critical for ensuring the data is clean, structured, and ready for subsequent steps.

3.2 DistilBERT Embeddings from tweet data

After cleaning the tweets, the next step involves generating embeddings using DistilBERT. This process translates the textual data into a numerical form that captures semantic meanings, which is used for further analysis.

The followings steps are done to generate the text embeddings:

- **Tokenization:** The process begins with tokenization, where the cleaned tweets are converted into a format that BERT can process. This involves breaking down the text into words or subwords, known as tokens, and mapping these tokens to their respective indices in DistilBERT’s vocabulary.
- **Generating embeddings:** The tokenized tweets from the previous step used as input for the DistilBERT model, it provides 768-size vectors (text embeddings) which provide the contextual of the text in a numerical form.
- **Aggregating Embeddings:** Since each tweet has its own embeddings, we combine them for each user. This is achieved by taking the mean of all tweets per user and getting a single embedding value for each user of size 768.

After completing the above steps, the generated embeddings are stored as a tensor file which we will later use for concatenation with GraphSage embeddings.

3.3 Feature Extraction from user data

In the context of this project, extracted features from user data are used as node features for GraphSage. This step involves pulling relevant numerical features from the cleaned user data directly and also deriving features from the user data. The following features are extracted from the user data:

- Tweet Count: Reflects the total activity level of the user, indicating how often they interact with the platform.
- Followers Count: Provides insight into the user’s social reach and popularity.
- Following Count: Helps assess the user’s networking behavior, which in excessive amounts can be indicative of spam or bot-like operations.
- Favorites Count: Shows the extent of the user’s engagement with content other than their own, giving a sense of how they interact with the platform.
- Account Age: Unlike the previous features which are readily available in the chosen datasets, this is derived from the creation date. The current date is subtracted from the user’s account creation date. The result is expressed days.

After extracting and calculating the necessary features, they are compiled into a structured tensor. This tensor aggregates the key numerical features for each user into a single, coherent format, which is then used as node features for GraphSage.

3.4 Graph Construction

Graph construction is a critical component in analyzing social networks, especially in tasks like bot detection where the relationships between entities (users) can provide significant insights. This process involves representing the social interactions between users as a graph, where nodes represent the users and edges represent the relationships or interactions between them.

The structure of the graph is defined by the edge index tensor file generated previously. This file contains information about the edges between nodes in a format that is optimized for processing with graph-based machine learning models.

The file is structured as a two-column tensor, where each column represents an edge. The first column contains the source nodes, and the second column contains the destination nodes. To further explain, consider a row $[1,0]$ and another one $[0,2]$.

This means there is an edge between users' 1 and 0 as well as users' 0 and 2. The below figure illustrates this idea.

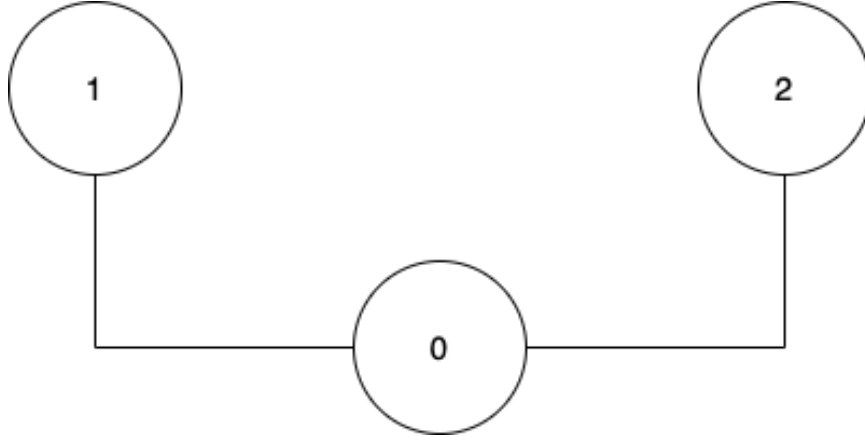


Figure 3: Graph Construction Example

The steps to construct the graph are as follows:

- Each user is represented as a node in the graph. The numerical properties extracted previously are used as node features.
- An edge is drawn between users as defined in the edge index matrix constructed in the data preprocessing stage.

The constructed graph serves as the input for the GraphSage model. The next step is to generate embeddings for each node.

3.5 GraphSage Embeddings

In this project, GraphSage is only used with the intention of generating embeddings for nodes. Hence, training epochs and optimization tasks are not required due to the lack of a prediction head. This enables a streamlined method to process the user relationship graph in a single, and reduces most of the complexity and overhead that is associated with graph based models.

The GraphSage model is initialized in the following manner:

- A single sageConv layer is used to aggregate the node neighborhood data, which

takes in 2 parameters namely `in_channels` and `out_channels`.

- `in_channels`: This corresponds to the number of node features used, which in this case is 5.
- `out_channels`: This corresponds to the size of the embeddings generated, which is set to 128. The size enables more detailed embeddings while maintaining compute efficiency.
- A graph data object is created using the `Data` class from PyTorch Geometric, which encapsulates the node features and the graph connectivity using the `edge_index` file.
- With the model in evaluation mode, embeddings are generated in a single forward pass. This operation does not require gradient calculations, as this process is purely for embedding generation rather than training.
- The `SAGEConv` layer processes the node features along with the graph structure to produce the final node embeddings. The embeddings are generated by aggregating features from each node’s immediate neighbors (and potentially their own features), transforming this aggregated information through the convolution layer to produce a new set of features (embeddings) for each node.
- The resulting GraphSAGE embeddings contain the transformed features for each user, encapsulating both their inherent attributes and the contextual information provided by their connections in the graph. These embeddings are concatenated with text embeddings to be used downstream for processing through an SVM.

These embeddings provides a snapshot of the node’s position and role within the broader network, effectively capturing both individual and collective characteristics.

3.6 Concatenation of Embeddings

In this project, we generate two different types of embeddings per user. One from their textual characteristics and the other from their network characteristics. By concatenating them side by side, it creates a comprehensive feature set to be used as an input for SVM.

The concatenation is done by basically stitching the GraphSage embedding (128-size) and the BERT embeddings (768-size) side-by-side to form a 896-dimensional vector. This is done for each user. This will be processed by an SVM classifier which is detailed in the next section.

Purpose of Concatenation:

- Concatenation allows embeddings to provide a more holistic view of the social network graph while incorporating the textual features, this results in a much richer feature set for SVMs. Since SVMs function quite well in high dimensionality [11], this strengthens the model even further.

3.7 SVM Classifier

From the previous steps, the concatenated 896-size embeddings are used as an input to the SVM classifier. The SVM classifier is initialized as follows:

- SVM was initialized using a linear kernel, this reduces the complexity and the computational overhead while providing high performance. Since we are dealing with potentially linearly separable high-dimensional data.
- The models performance was optimized by opting to not use a kernel trick, but rather relying on the inherent separability in this transformed space.
- The regularization term was appropriately adjusted to avoid overfitting.
- 5 fold cross-validation technique was employed to enable a robust training and testing procedure for the model.

Cross-validation was a quite important step to improve the overall model accuracy while also reducing skews introduced by any unbalanced subset.

3.8 Evaluation

All the experiments conducted were evaluated using standard metrics such as Accuracy and F1 scores. The results of the experiments were compared to different baselines and state-of-the-art models on each dataset. The results of the experiments are tabulated in the next section.

3.8.1 Execution Details

This section highlights the some of the key execution details for training each of models as listed in the experiments.

Method	Dataset	Training Time (hours:mins)
BERT+GraphSage	Cresci-15	3:47
DistilBERT+GraphSage	Cresci-15	2:32
GraphSage+SVM	Twibot-22	10:55
DistilBERT+GraphSage	Twibot-22	14:29
BERT+GraphSage*	Twibot-22	300>

Table 3: Training time comparison of different methods

From the above table we can observe that BERT+GraphSage* (Includes preprocessing and training time) requires more than 300 hours for the Twibot dataset if all tweets of all accounts are used. Since execution for such a long duration is infeasible, constraints to limit number of tweets per user were added along with faster BERT models such as DistilBERT.

All the experiments were conducted on a MacBook Pro 2019 with an 2.4 GHz Quad-Core Intel Core i5 Processor and 8 GB 2133 MHz LPDDR3 of RAM.

CHAPTER 4

Results

4.1 Results

This section showcases the results for each experiment on the Twibot-22 and Cresci-15 datasets. The results seem promising despite the challenges encountered for the scale and timeline of the project. The idea of concatenating embeddings to form an enhanced feature set is shown to provide value.

4.1.1 Cresci-15 Results

The table below shows the results of the experiments along with a comparison with other baselines. This methodology outperforms all other models on the Cresci-15 dataset.

Method	Type (F/G)	Accuracy (%)	F1 score (%)
SATAR [12]	F	93.42	95.05
GCN [13]	G	77.08	77.91
BotRGCN [3]	G	96.52	97.30
RGT [12]	G	97.15	97.78
BIC [12]	G	98.35	98.71
GraphSage+BERT	G	98.68	98.92
GraphSage+DistilBERT	G	98.56	98.88

Table 4: Performance comparison of different methods on the Cresci dataset

The approach using GraphSage combined with BERT and DistilBERT embeddings yields high performance, as evidenced by accuracy and F1 scores:

- GraphSage+BERT achieves an accuracy of 98.68% and an F1 score of 98.92%.
- GraphSage+DistilBERT records slightly lower metrics, with an accuracy of 98.56% and an F1 score of 98.88%.

Both configurations surpass the performance of traditional graph-based models such as GCN, which has an accuracy and F1 score of 77.08% and 77.91%, respectively. Notably, the models also exhibit competitive or superior performance compared to

other recent methods:

- BotRGCN reports an accuracy of 96.52% and an F1 score of 97.30
- RGT shows an accuracy of 97.15% and an F1 score of 97.78%, and
- BIC presents the highest scores among other compared models with an accuracy of 98.35% and an F1 score of 98.71%.

The improvement in accuracy and F1 score in the models can be attributed to the effective combination of GraphSage’s capability to capture neighborhood information and BERT’s sophisticated handling of textual features. While GraphSage+DistilBERT offers a small reduction in performance relative to GraphSage+BERT, it provides a beneficial trade-off between efficiency and effectiveness, leveraging the reduced complexity of DistilBERT.

4.1.2 Twibot-22 Results

For Twibot 22, the model was implemented twice, once without any BERT embeddings using only the GraphSage embeddings and once with DistilBERT embeddings. The models GraphSage+SVM and GraphSage+DistilBERT are compared against other contemporary approaches as presented in the below Table. Each model was evaluated using a 5-fold cross-validation method to ensure the integrity and comparability of the results.

Method	Type (F/G)	Accuracy (%)	F1 score (%)
SVM [14]	F	49.30	-
GCN [15]	G	47.72	38.10
HGT [16]	G	74.91	39.60
BotRGCN [3]	G	79.66	57.50
RGT [9]	G	76.47	49.24
BGSRD [1]	G	71.88	21.14
GraphSage+SVM	G	67.21	48.32
GraphSage+DistilBERT	G	74.62	51.69

Table 5: Performance comparison of different methods on the Twibot dataset

The performance of the models in question, based on accuracy and F1 scores, is as follows:

- GraphSage+SVM showed an accuracy of 67.21% and an F1 score of 48.32%.
- GraphSage+DistilBERT delivered an accuracy of 74.62% and an F1 score of 51.69%.

Comparing and contrasting these results against other notable methods tested on the same dataset:

- SVM alone had a notably lower accuracy of 49.30%. F1 score was not reported.
- GCN and HGT delivered accuracy scores of 47.72% and 74.91% respectively, with F1 scores at 38.10% and 39.60%.
- BotRGCN and RGT performed better, with BotRGCN achieving an accuracy of 79.66% and an F1 score of 57.50%, while RGT reported an accuracy of 76.47% and an F1 score of 49.24%.
- BGSRD presented an accuracy of 71.88% but a much lower F1 score of 21.14%.

The incorporation of GraphSage with DistilBERT significantly improved both accuracy and F1 scores over using GraphSage with a traditional SVM classifier. This suggests that the textual context captured by DistilBERT embeddings provides meaningful enhancement over the graph structural information alone, as provided by GraphSage.

CHAPTER 5

Conclusion and Future Work

This project aimed to explore and enhance the effectiveness of bot detection methodologies on social media platforms, specifically focusing on the Cresci-15 and Twibot-22 datasets. By integrating GraphSage with both BERT and its distilled version, DistilBERT, the developed models have shown to outperform all traditional models while also competing with most and outperforming some of the existing state-of-the-art methods.

On the Cresci-15 dataset, the GraphSage+BERT model demonstrated exceptional performance achieving an which was amongst the highest recorded on this dataset. This underscores the effectiveness of combining rich textual features extracted by BERT with the structural insights provided by GraphSage.

Although the GraphSage+DistilBERT model slightly underperformed compared to its BERT counterpart, it offered a significant computational advantage. This model's balance between efficiency and performance makes it particularly suitable for scenarios where computational resources are constrained.

The trends observed on the Cresci-15 dataset were mirrored on the Twibot-22 dataset, albeit with overall lower performance metrics across all models. This was expected due to the inherent challenges and complexities of the Twibot-22 dataset. Even here, the GraphSage+DistilBERT model stood out, providing a good balance between accuracy and computational demand. By contrasting with the GraphSage+SVM only model, it also demonstrated that the addition of the DistilBERT embeddings greatly increased the accuracy.

The project successfully demonstrated the integration of GraphSage with BERT embeddings presents a robust approach to bot detection on social media. This approach effectively harnesses the synergy between advanced natural language processing

techniques and graph-based learning algorithms to address the intricate challenges of identifying inauthentic behavior across diverse social media platforms. The exploration of both full and distilled versions of BERT within this framework offers valuable insights into balancing computational efficiency with performance, making this methodology adaptable for various operational environments and resource constraints.

5.1 Future Work

To further enhance the efficacy and applicability of the bot detection models developed in this project, several areas of future research are recommended:

As observed, the limitation to the 15 most recent tweets per user due to computational constraints may restrict the model’s learning potential. If computational limitations are not a concern, all tweets of each user should be considered. This could potentially lead to significantly better results.

While DistilBERT offers a balance between performance and efficiency, employing the full BERT model has a huge chance improve the detection capabilities due to its richer linguistic understanding. This phenomenon is already observed on the Cresci-15 dataset.

Another improvement that can be made is by modifying the GraphSage model and including even more node features. Current implementation only uses 5 numerical features, more numerical and non-numerical features can also be tested to evaluate the feature set.

The idea of concatenating embeddings can be applied to others areas such as detecting fake news and combating misinformation. In the case [17], they employ GraphSage to detect Fake news. Integration of text based features using BERT has a chance to significantly improve upon the findings presented there.

LIST OF REFERENCES

- [1] Q. Guo, H. Xie, Y. Li, W. Ma, and C. Zhang, “Social bots detection via fusing bert and graph convolutional networks,” *Symmetry*, vol. 14, no. 1, 2022. [Online]. Available: <https://www.mdpi.com/2073-8994/14/1/30>
- [2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” 2018. [Online]. Available: <https://arxiv.org/abs/1710.10903>
- [3] S. Feng, H. Wan, N. Wang, and M. Luo, “Botrgcn: Twitter bot detection with relational graph convolutional networks,” in *Proceedings of the 2021 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining*, ser. ASONAM ’21. New York, NY, USA: Association for Computing Machinery, 2022, p. 236–239. [Online]. Available: <https://doi.org/10.1145/3487351.3488336>
- [4] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS’17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 1025–1035.
- [5] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *International Conference on Learning Representations*, 2017. [Online]. Available: <https://openreview.net/forum?id=SJU4ayYgl>
- [6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Minneapolis, Minnesota: Association for Computational Linguistics, June 2019, pp. 4171–4186. [Online]. Available: <https://aclanthology.org/N19-1423/>
- [7] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter,” 2020. [Online]. Available: <https://arxiv.org/abs/1910.01108>
- [8] S. Feng, H. Wan, N. Wang, J. Li, and M. Luo, “Twibot-20: A comprehensive twitter bot detection benchmark,” in *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, ser. CIKM ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 4485–4494. [Online]. Available: <https://doi.org/10.1145/3459637.3482019>

- [9] S. Feng, Z. Tan, H. Wan, N. Wang, Z. Chen, B. Zhang, Q. Zheng, W. Zhang, Z. Lei, S. Yang, X. Feng, Q. Zhang, H. Wang, Y. Liu, Y. Bai, H. Wang, Z. Cai, Y. Wang, L. Zheng, Z. Ma, J. Li, and M. Luo, “Twibot-22: Towards graph-based twitter bot detection,” in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 35 254–35 269. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/e4fd610b1d77699a02df07ae97de992a-Paper-Datasets_and_Benchmarks.pdf
- [10] M. Mazza, S. Cresci, M. Avvenuti, W. Quattrociocchi, and M. Tesconi, “Rtbust: Exploiting temporal patterns for botnet detection on twitter,” in *Proceedings of the 10th ACM Conference on Web Science*, ser. WebSci ’19. New York, NY, USA: Association for Computing Machinery, 2019, p. 183–192. [Online]. Available: <https://doi.org/10.1145/3292522.3326015>
- [11] Y. Yang, J. Li, and Y. Yang, “The research of the fast svm classifier method,” in *2015 12th International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP)*, 2015, pp. 121–124.
- [12] Z. Lei, H. Wan, W. Zhang, S. Feng, Z. Chen, J. Li, Q. Zheng, and M. Luo, “Bic: Twitter bot detection with text-graph interaction and semantic consistency,” 2023. [Online]. Available: <https://arxiv.org/abs/2208.08320>
- [13] Y. Wu, D. Lian, Y. Xu, L. Wu, and E. Chen, “Graph convolutional networks with markov random field reasoning for social spammer detection,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 01, pp. 1054–1061, Apr. 2020. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/5455>
- [14] J. Rodríguez-Ruiz, J. I. Mata-Sánchez, R. Monroy, O. Loyola-González, and A. López-Cuevas, “A one-class classification approach for bot detection on twitter,” *Computers Security*, vol. 91, p. 101715, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404820300031>
- [15] S. Ali Alhosseini, R. Bin Tareaf, P. Najafi, and C. Meinel, “Detect me if you can: Spam bot detection using inductive representation learning,” in *Companion Proceedings of The 2019 World Wide Web Conference*, ser. WWW ’19. New York, NY, USA: Association for Computing Machinery, 2019, p. 148–153. [Online]. Available: <https://doi.org/10.1145/3308560.3316504>
- [16] Z. Hu, Y. Dong, K. Wang, and Y. Sun, “Heterogeneous graph transformer,” in *Proceedings of The Web Conference 2020*, ser. WWW ’20. New York, NY, USA: Association for Computing Machinery, 2020, p. 2704–2710. [Online]. Available: <https://doi.org/10.1145/3366423.3380027>

- [17] V.-H. Nguyen, K. Sugiyama, P. Nakov, and M.-Y. Kan, “Fang: Leveraging social context for fake news detection using graph representation,” in *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, ser. CIKM '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1165–1174. [Online]. Available: <https://doi.org/10.1145/3340531.3412046>

APPENDIX

Generating Tweet embeddings using DistilBERT

```
1 import torch
2 from tqdm import tqdm
3 import numpy as np
4 from transformers import pipeline
5 import pandas as pd
6 import json
7 import os
8
9 # Load user data (adjust the path if necessary)
10 user = pd.read_json('../datasets/Twibot-22/user.json')
11
12 # Load tweets data, assuming each entry contains timestamp info
13 each_user_tweets = json.load(open("processed_data/
    twibot_id_tweet.json", 'r'))
14
15 # Initialize BERT feature extraction pipeline
16 feature_extract = pipeline('feature-extraction', model='
    distilbert-base-uncased', tokenizer='distilbert-base-uncased
    ', device=1, padding=True, truncation=True, max_length=50,
    add_special_tokens=True)
17
18 def tweets_embedding():
19     print('Running tweet embeddings generation')
20     path = "../processed_data/twibot_tweets_tensor.pt"
```

```

21     if not os.path.exists(path):
22         tweets_list = []
23     for i in tqdm(range(len(each_user_tweets))):
24         user_tweets = each_user_tweets.get(str(i), [])
25         if not user_tweets:
26             total_each_person_tweets = torch.zeros(768) #
                BERT base produces 768-dimensional
                embeddings
27         else:
28             # Sort tweets by timestamp and select the most
                recent 15
29             user_tweets = sorted(user_tweets, key=lambda x:
                x['timestamp'], reverse=True)[:15]
30             tweet_tensors = []
31             for tweet in user_tweets:
32                 if tweet['text'] is None:
33                     tweet_tensor = torch.zeros(768)
34                 else:
35                     tweet_embeddings = torch.tensor(
                        feature_extract(tweet['text']))
36                     # Average embeddings across the tweet
37                     tweet_tensor = torch.mean(
                        tweet_embeddings.squeeze(0), dim=0)
38                     tweet_tensors.append(tweet_tensor)
39
40             # Average over all tweets to get the user's
                tweet embedding

```

```

41         total_each_person_tweets = torch.mean(torch.
           stack(tweet_tensors), dim=0)
42
43         tweets_list.append(total_each_person_tweets)
44
45         # Save all user tweet embeddings to a tensor
46         tweet_tensor = torch.stack(tweets_list)
47         torch.save(tweet_tensor, path)
48     else:
49         tweet_tensor = torch.load(path)
50     print('Finished')
51
52 tweets_embedding()

```

Listing A.1: DistilBERT implementation limiting number of tweets per user to 15